

Chapter 4

The Processor

Hazards

- Situations that prevent starting the next instruction in the next cycle
- Structure hazards
 - A required resource is busy
- Data hazard
 - Need to wait for previous instruction to complete its data read/write
- Control hazard
 - Deciding on control action depends on previous instruction

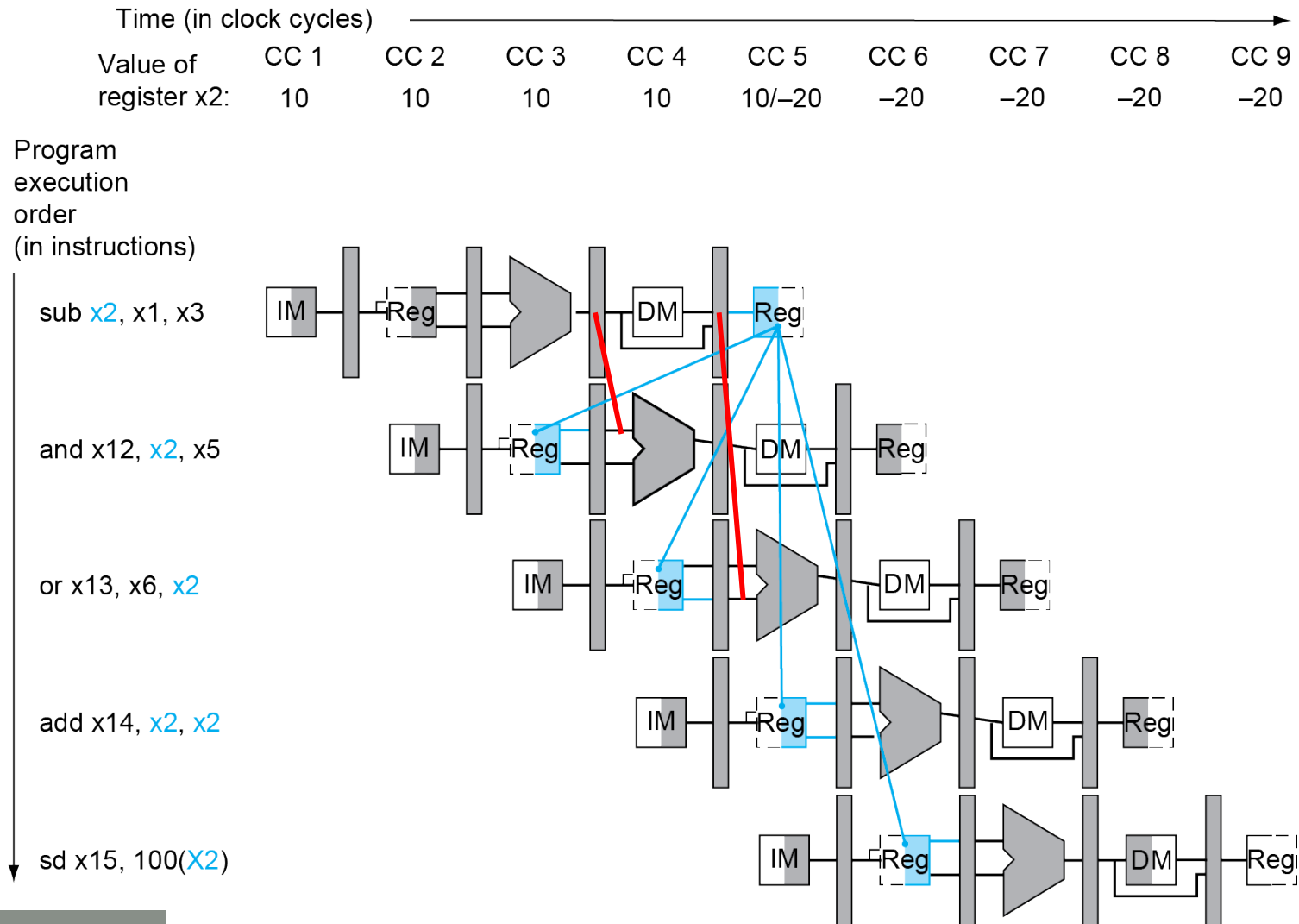
Data Hazards in ALU Instructions

- Consider this sequence:

```
sub    x2, x1, x3
and    x12, x2, x5
or     x13, x6, x2
add    x14, x2, x2
sd     x15, 100(x2)
```

- We can resolve hazards with forwarding
 - How do we detect when to forward?

Dependencies & Forwarding



Detecting the Need to Forward

- Pass register numbers along pipeline
 - e.g., ID/EX.RegisterRs1 = register number for Rs1 sitting in ID/EX pipeline register
- ALU operand register numbers in EX stage are given by
 - ID/EX.RegisterRs1, ID/EX.RegisterRs2
- Data hazards when
 - 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1
 - 1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2
 - 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1
 - 2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

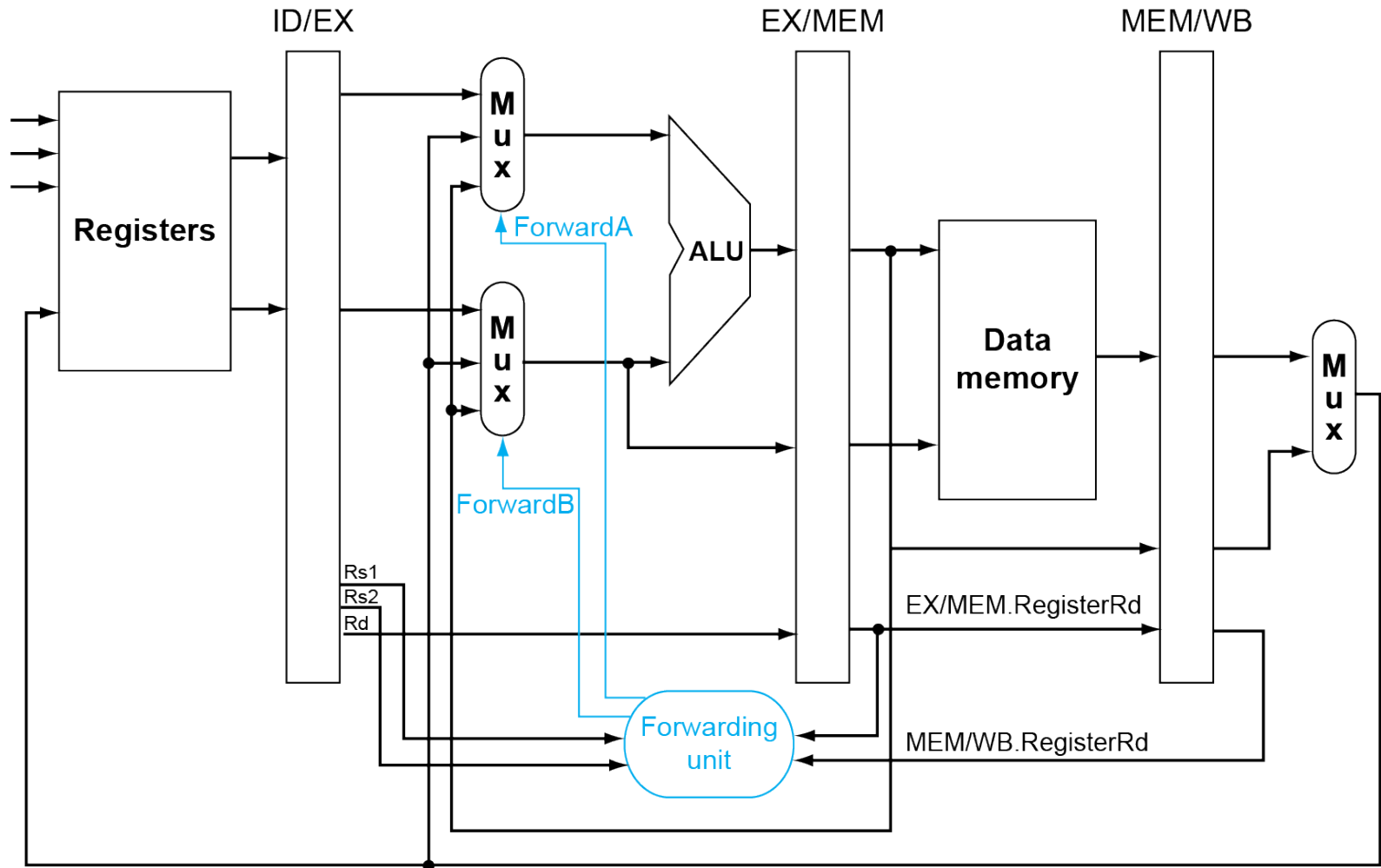
Fwd from
EX/MEM
pipeline reg

Fwd from
MEM/WB
pipeline reg

Detecting the Need to Forward

- But only if forwarding instruction will write to a register!
 - EX/MEM.RegWrite, MEM/WB.RegWrite
- And only if Rd for that instruction is not x0
 - EX/MEM.RegisterRd $\neq$ 0,
MEM/WB.RegisterRd $\neq$ 0

Forwarding Paths



Forwarding Conditions

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

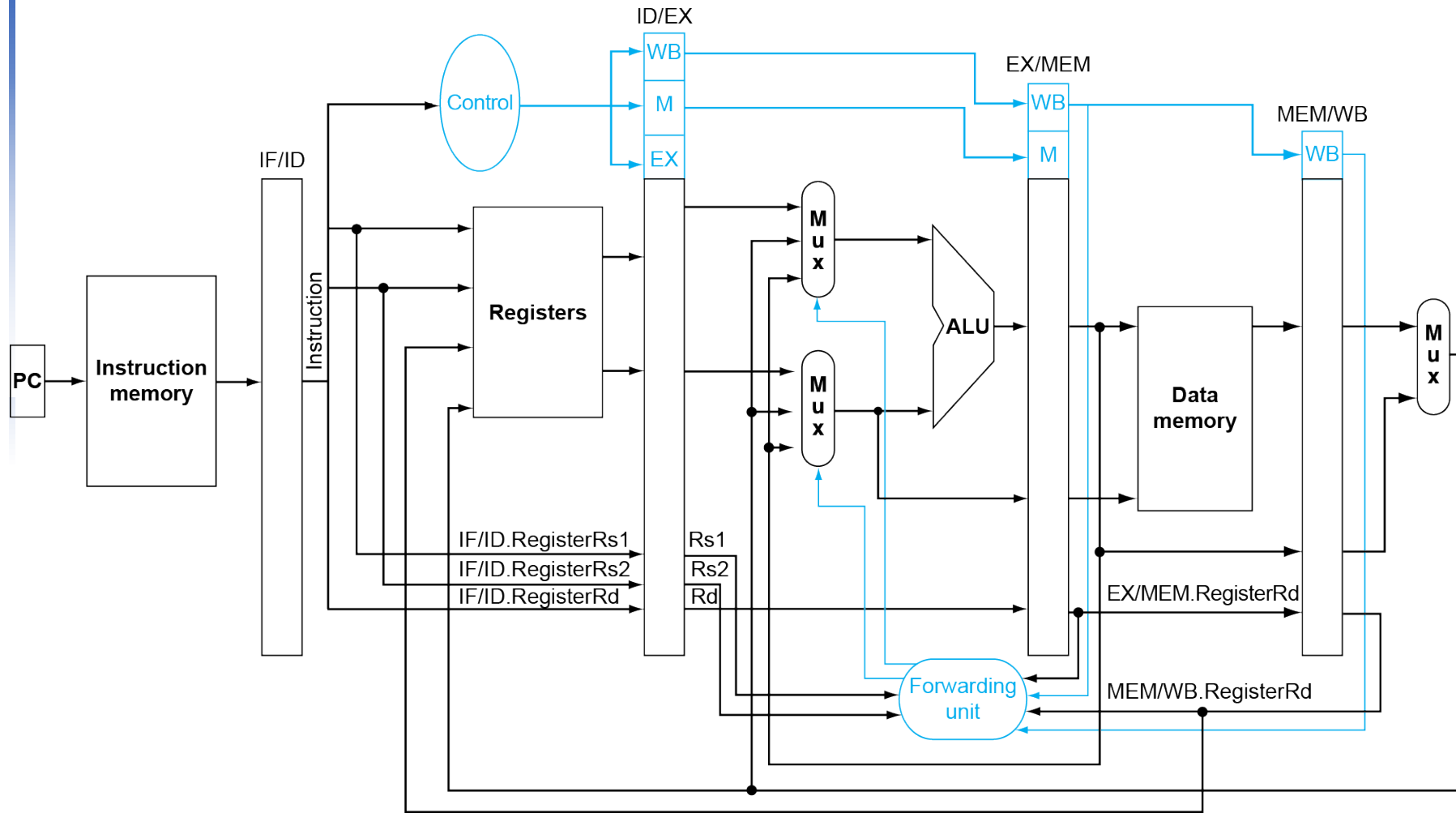
Double Data Hazard

- Consider the sequence:
 add x1, x1, x2
 add x1, x1, x3
 add x1, x1, x4
- Both hazards occur
 - Want to use the most recent
- Revise MEM hazard condition
 - Only fwd if EX hazard condition isn't true

Revised Forwarding Condition

- MEM hazard
 - if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd $\neq$ ID/EX.RegisterRs1))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01
 - if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd $\neq$ ID/EX.RegisterRs2))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

Datapath with Forwarding



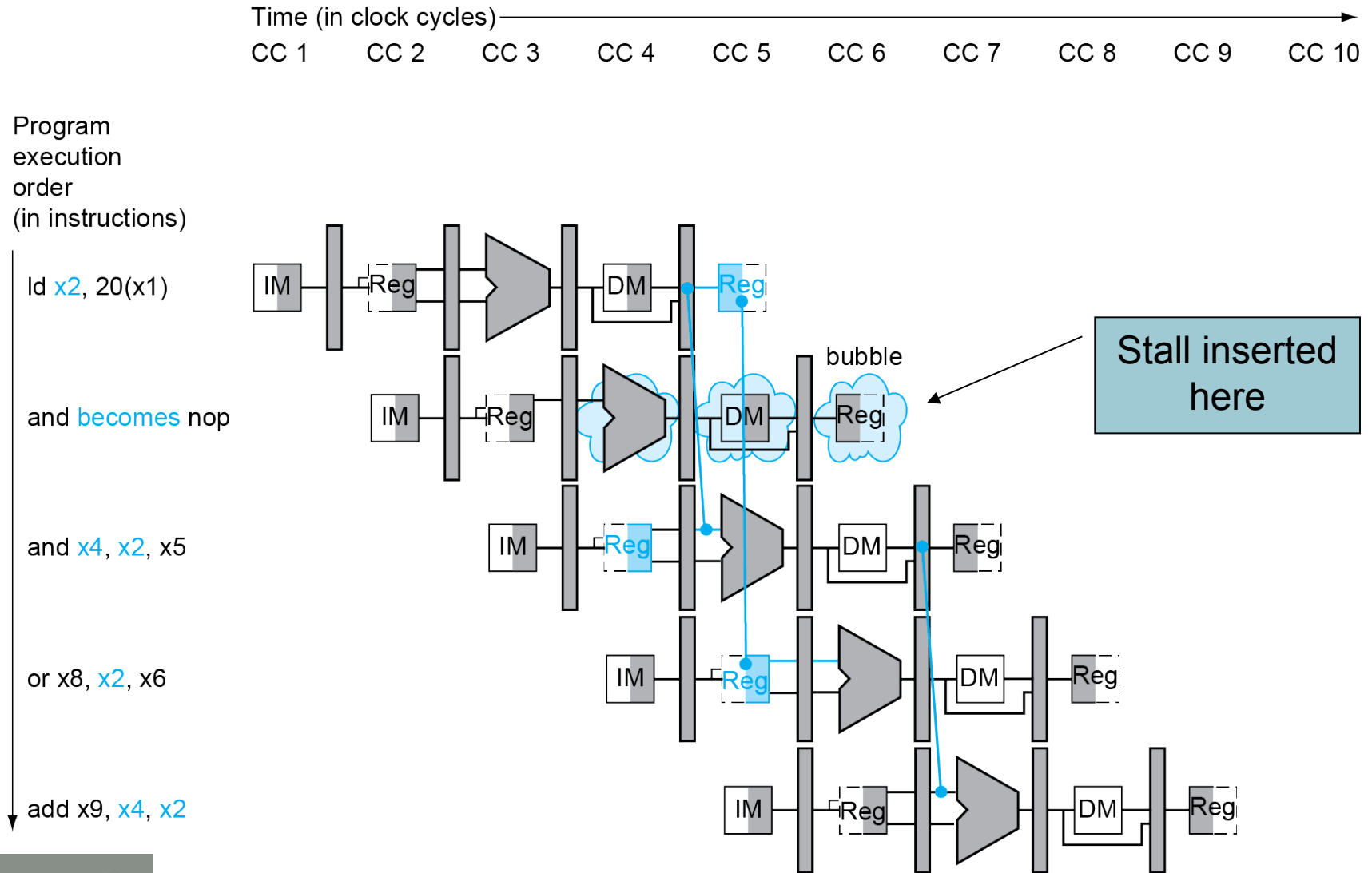
Load-Use Hazard Detection

- Check when using instruction is decoded in ID stage
- ALU operand register numbers in ID stage are given by
 - IF/ID.RegisterRs1, IF/ID.RegisterRs2
- Load-use hazard when
 - ID/EX.MemRead and
((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
(ID/EX.RegisterRd = IF/ID.RegisterRs2))
- If detected, stall and insert bubble

How to Stall the Pipeline

- Force control values in ID/EX register to 0
 - EX, MEM and WB do nop (no-operation)
- Prevent update of PC and IF/ID register
 - Using instruction is decoded again
 - Following instruction is fetched again
 - 1-cycle stall allows MEM to read data for `ld`
 - Can subsequently forward to EX stage

Load-Use Data Hazard



MK



Stalls and Performance

The BIG Picture

- Stalls reduce performance
 - But are required to get correct results
- Compiler can arrange code to avoid hazards and stalls
 - Requires knowledge of the pipeline structure